

Scenariul Prezentării Academice

Modulul: Metode și Practici în Informatică (MPI)

David Tudor

Universitatea de Vest din Timișoara (UVT)

Facultatea de Informatică

david.tudor06@e-uvt.ro

Mai 2026

1 Analiza Audienței

În conformitate cu cerințele riguroase de proiectare a discursului academic, audiența a fost evaluată pe baza următoarelor criterii:

- **Cui ne adresăm:** Profesorul evaluator și colegii de grupă din cadrul Facultății de Informatică, UVT.
- **Nivelul de cunoștințe:** Ridicat (studenți și experți în domeniu), stăpânind noțiunile fundamentale de programare C++ și analiză asimptotică.
- **Așteptări:** Analiză centrată pe date empirice, explicarea clară a anomaliilor hardware/software și evitarea redundanțelor teoretice.

2 Formularul de Derulare a Prezentării (Scenariul Oficial)

Faza	Obiective operaționale	Metoda	Text	Materiale suport	Timp	Observații organizatoriale
Motivație (Intro)	Captarea atenției; stârnirea interesului academic.	Expunere, Întrebare retorică.	Bună ziua! Mă numesc David Tudor, iar astăzi vă voi prezenta un studiu comparativ asupra algoritmilor de sortare. De ce am făcut acest studiu? Pentru a evidenția decalajul dintre teoria asimptotică și practică, precum și influența arhitecturii hardware asupra performanței reale a codului.	Slide 1: Titlu Slide 2: Intro & Motivație	2.5 min	Verificarea proiectorului și a lucrării. Menținerea contactului vizual.
Învățare (Arhitectură)	Identificarea structurii modulare a aplicației de test.	Expunere descriptivă.	Pentru a realiza testele, am dezvoltat o aplicație în C++, secționată în trei componente distincte: <code>Algorithm.h</code> pentru logica de sortare, <code>Generators.h</code> pentru distribuții și <code>Benchmark.h</code> , care folosește biblioteca <code><chrono></code> pentru a capta timpii exacti în milisecunde.	Slide 3: Arhitectura Codului	3 min	Vorbire clară, ritm rar și articulat. Indicarea modulelor direct pe ecran.
Învățare (Rezultate)	O1: Identificarea pragului critic de ineficiență a algoritmilor $O(n^2)$.	Analiză critică, Expunere date.	Privind rezultatele brute pe setul de 10.000 de elemente, tabelul evidențiază o prăpastie uriașă: pentru date aleatorii, Bubble Sort are nevoie de peste 650 ms, în timp ce Quick Sort finalizează în doar 1.89 ms. Însă cifrele se schimbă spectaculos dacă analizăm ferestrele de scalabilitate.	Slide 4: Tabel de Date Slide 5: Grafice de scalabilitate	4 min	Orientarea atenției audienței către rezultate.
Asociere (Anomalii)	O2: Explicarea anomaliilor (Quick Sort pe liste constante și Insertion Sort).	Dialog ghidat, Demonstrație (Reintrare).	Cel mai important aspect a fost descoperirea anomaliilor Prima: Quick Sort a căzut masiv pe liste constante (47.39 ms) din cauza pivotului. A doua: Insertion Sort a obținut un timp uluitor de 0.08 ms pe liste deja sortate, depășind toți algoritmii moderni.	Slide 6: Analiza Anomaliilor (Grafice comparative)	2.5 min	Schimbarea tonului vocii pentru a marca elementul inedit al cercetării.
Asociere (Sinteză)	O3: Selectarea algoritmului optim în funcție de structura datelor.	Sinteză, Dialog deschis.	În concluzie, testele mele demonstrează că nu există un algoritm universal suprem. Optimizarea depinde direct de context și de datele de intrare. Vă mulțumesc pentru atenție și vă invit să deschidem sesiunea de întrebări și discuții!	Slide 7: Concluzii și final	3 min	Deschiderea dialogului activ cu profesorul evaluator și colectarea întrebărilor. Răspunsuri calme.